MANUEL PÉDAGOGIQUE D'APPRENTISSAGE & SOUTENANCE L3

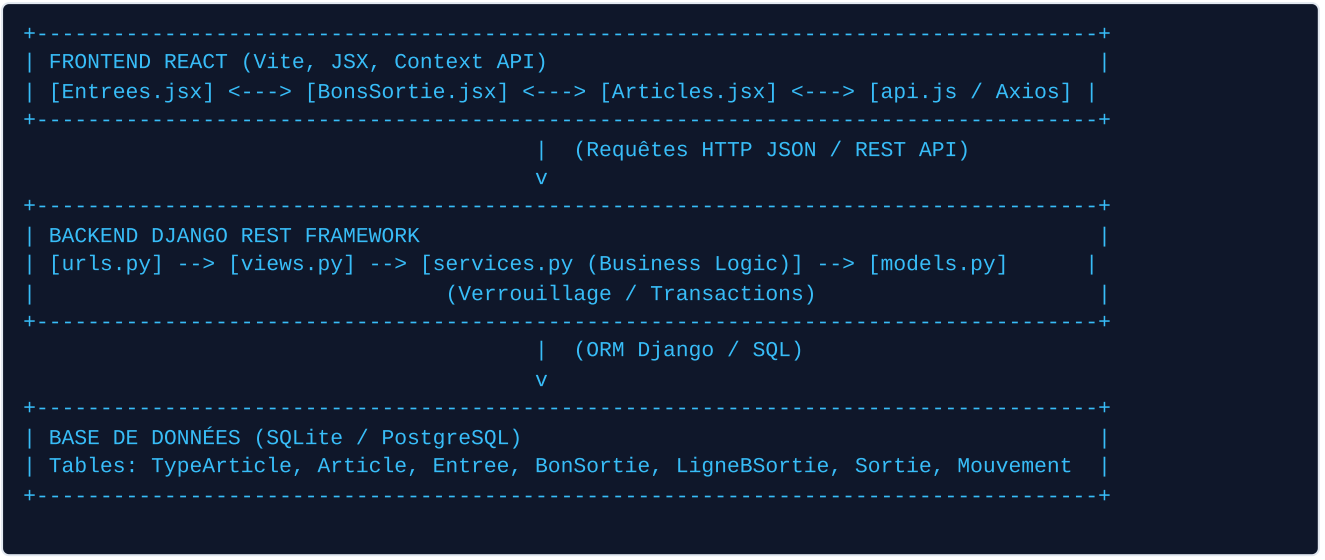
PAOSITRA SMART ASSET

Compréhension approfondie du code source, workflow et architecture

Objectif de ce manuel : Destiné à l'étudiante en informatique L3 pour maîtriser l'intégralité du code source du projet. Ce document décortique l'architecture backend Django REST Framework et frontend React, le modèle de données, le workflow multi-acteurs et fournit les clés de réponse pour la soutenance.

1. Architecture Générale & Flux Découplé

L'application **PAOSITRA SMART ASSET** repose sur une architecture découplée (SPA Client / Serveur API REST). Le backend Django gère la persistance, l'étanchéité des transactions métier et la sécurité des rôles. Le frontend React assure une interface réactive avec gestion d'état locale et modales de traçabilité.



2. Technologies et Rôles dans le Projet

Technologie	Domaine	Rôle dans PAOSITRA SMART ASSET
Django 4.x & DRF	Backend API	Gestion de la base de données, ORM, règles métier, transactions atomiques, serializers.
React 18 & Vite	Frontend SPA	Interface utilisateur dynamique, composants formulaires, modales d'audit, hooks d'état.
Atomic Transactions	Concurrence DB	`transaction.atomic()` garantit que tout échec annule l'intégralité des écritures en BD.
Pessimistic Locking	Sécurité Stock	`select_for_update()` bloque les lignes de stock en BD durant le traitement d'un Bon de Sortie.

3. Analyse du Backend — Modèles de Données (`models.py`)

La modélisation distingue fondamentalement deux types de biens : les matériels **individualisés** (ordinateurs, imprimantes avec numéros de série et QR codes) et les matériels **consommables** (ramettes de papier, cartouches gérées uniquement en quantité globale).

Structure du Stock (Entities)

TypeArticle : Famille de matériel, possède `quantite_stock` et l'indicateur `gestion_individuelle`. **Article :** Exemplaire physique unique. Lié à une `LigneEntree` et possède `ref_article`.

Workflow Bon de Sortie

BonSortie : En-tête de demande (statuts: brouillon, en_attente_autorisation, en_attente_validation, valide, refuse, sorti). **LigneBSortie :** Quantités demandée, autorisée, validée et `etat_ligne`.

4. Cœur Méthodologique — Le Fichier `services.py`

Toutes les opérations d'écriture en base vivent exclusivement dans `services.py`. Aucune modification de stock ou de statut n'est exécutée directement dans les vues ou les formulaires.

```
# 1. Calcul dynamique du Stock Réservé (Non stocké en BD)
def stock_reserve(type_article: TypeArticle) -> int:
    lignes = LigneBSortie.objects.filter(type_article=type_article)
    autorisees = lignes.filter(etat_ligne='autorisee').aggregate(t=Sum('quantite_autorisee'))['t']
or 0
    validees = lignes.filter(etat_ligne='validee').aggregate(t=Sum('quantite_validee'))['t'] or 0
    return autorisees + validees

# 2. Vérification de disponibilité réelle pour le DSID
def stock_disponible_pour_autorisation(type_article: TypeArticle) -> int:
    return type_article.quantite_stock - stock_reserve(type_article)
```

Règle d'or de Soutenance (Stock Réservé) : Le stock réservé n'est PAS un champ dans la table `TypeArticle`. S'il était stocké en base, une panne ou une interruption lors d'une requête risquerait de désynchroniser le stock réel du stock réservé. Il est donc calculé dynamiquement à la volée.

5. Verrouillage Pessimiste et Antidote à la Concurrence

Si deux agents tentent de valider un Bon de Sortie au même instant sur le même matériel, le système utilise `select_for_update()` pour sérialiser les transactions.

```
def _verrouiller_type(type_article):
    """Relit le type en base avec un verrou SQL 'SELECT ... FOR UPDATE'"""
    return TypeArticle.objects.select_for_update().get(pk=type_article.pk)
```

6. Le Workflow Multi-Acteurs du Bon de Sortie

Le passage d'un Bon de Sortie s'effectue à travers 4 étapes étanches, contrôlées par le système de rôles utilisateur :

```
           [DEMANDEUR] ---> Soumet le BS (Statut: brouillon -> en_attente_autorisation)
                                                  |
                                                  v
            [AGENT DSID] ---> Autorise chaque ligne (quantite_autorisee, stock réservé)
                |            (Statut: en_attente_autorisation -> en_attente_validation)
                                                  v
              [AGENT DCA]  ---> Valide définitivement les quantités (quantite_validee)
                           |            (Statut: en_attente_validation -> valide)
                                                  v
      [AGENT CAME]  ---> Exécute la Sortie physique & décrémente TypeArticle.quantite_stock !
                                          (Statut: valide -> sorti)
```

7. Décortiquage de la Fonction `autoriser_ligne()`

```
def autoriser_ligne(ligne, utilisateur, quantite, motif=''):
    _verifier_role(utilisateur, ROLES_AUTORISATION, "autoriser une ligne")
    motif = (motif or '').strip()
    with transaction.atomic():
        bs = BonSortie.objects.select_for_update().get(pk=ligne.bon_sortie_id)
        ligne_db = LigneBSortie.objects.select_for_update().get(pk=ligne.pk)

        if ligne_db.etat_ligne != 'demandee':
            raise ValidationError("Cette ligne a déjà été traitée par le DSID.")

        if quantite > 0:
            type_verrouille = _verrouiller_type(ligne_db.type_article)
            disponible = type_verrouille.quantite_stock - stock_reserve(type_verrouille)
            if quantite > disponible:
                raise ValidationError(f"Quantité {quantite} > stock disponible {disponible}")

        Intervient.objects.create(
            ligne_bsortie=ligne_db, utilisateur=utilisateur, role_intervention='autorisation',
            quantite_decidee=quantite, motif=motif or None
        )
        ligne_db.quantite_autorisee = quantite
        ligne_db.etat_ligne = 'autorisee' if quantite > 0 else 'refusee'
        ligne_db.save(update_fields=['quantite_autorisee', 'etat_ligne'])
        _recalculer_statut(bs)
    return ligne_db
```

Analyse pas à pas :

1. `_verifier_role()` rejette immédiatement les utilisateurs sans privilèges DSID.
2. `transaction.atomic()` isole la transaction SQL. Si une erreur survient, la BD revient à l'état initial.
3. `_recalculer_statut(bs)` vérifie si toutes les lignes du bon ont été traitées. Quand la dernière ligne est validée/refusée, l'en-tête du bon passe automatiquement à l'état suivant.

8. Analyse du Frontend React (`BonsSortie.jsx` & `Articles.jsx`)

Le frontend s'appuie sur la bibliothèque Axios (via le wrapper `api.js`) pour requêter le backend Django REST. L'état local React (avec `useState`, `useMemo`, `useLoad`) réagit dynamiquement aux interactions.

Gestion Intelligente de l'Autorisation DSID avec Garde-Fou Stock

Afin d'éviter de bloquer le workflow lorsqu'un article est en rupture de stock (stock = 0), le frontend charge en amont le stock physique et interroge l'utilisateur avec validation continue :

```
// Extrait de BonsSortie.jsx - Prompt avec vérification dynamique du stock
const typesArticles = await api.list('types-article');
for (const ligne of lignesDemandees) {
  const typeInfo = (typesArticles || []).find(t => String(t.id) === String(ligne.type_article));
  const stockActuel = typeInfo ? typeInfo.quantite_stock : 0;

  while (true) {
    const messagePrompt = `[STOCK DISPONIBLE : ${stockActuel}]\nArticle : ${ligne.type_article_nom}
\n...`;
    const qteSaisie = prompt(messagePrompt, Math.min(ligne.quantite_demandee, stockActuel));
    if (qteSaisie === null) return;

    const quantite = parseInt(qteSaisie, 10);
    if (quantite > stockActuel) {
      alert(`Erreur : La quantité autorisée (${quantite}) ne peut pas dépasser le stock physique
(${stockActuel}).`);
      continue; // Boucle de ressaisie
    }
    await api.autoriser(ligne.id, quantite, motif);
    break;
  }
}
```

9. Traçabilité Unitaire des Articles (`Articles.jsx`)

Dans la vue des Articles physiques, un clic sur la référence d'un article ouvre une modale de traçabilité interrogeant le journal des mouvements.

Logique de Traçabilité au Clic :
Lorsqu'un utilisateur clique sur `ref_article`, la fonction `ouvrirHistoriqueArticle(article)` filtre la liste globale des `mouvements` sur l'ID de cet article spécifique. Si un mouvement de type `sortie` existe, la modale affiche la date et le nom de l'entité destinataire. Dans le cas contraire, un badge confirme que l'article est actuellement **En Stock**.

10. Parcours Complet d'une Opération en Base

Suivons pas à pas le parcours d'une transaction, depuis le clic de l'agent CAME sur "Effectuer Sortie" jusqu'à la mise à jour de la base de données SQLite/PostgreSQL.

Étape	Composant Executeur	Action Technologique / Transaction
1. Clic Interface	React (`BonsSortie.jsx`)	L'utilisateur déclenche `effectuerSortie()`. L'API transmet le payload JSON avec l'ID du bon et la date.
2. Routing HTTP	Django (`urls.py` -> `views.py`)	La requête est réceptionnée par `SortieViewSet.create()`. La vue vérifie les droits du rôle `agent_came`.
3. Transaction Atomic	Service (`services.py`)	Ouverture du bloc `with transaction.atomic()`. Blocage exclusif des enregistrements avec `select_for_update()`.
4. Traitement Stock	Service (`services.py`)	Pour les articles individualisés, sélection des instances `Article` non encore sorties, génération des `Mouvement` et décrémentation de `TypeArticle.quantite_stock`.
5. Commit BD	Base de Données	Validation définitive du Commit SQL. En cas d'exception, ROLLBACK automatique.
6. Refresh UI	React (`BonsSortie.jsx`)	Promesse JSON résolue (HTTP 201 Created). Execution de `reload()`, réaffichage des cartes et statuts mis à jour.

> **Distinction Pédagogique Majeure (Entrée vs Sortie) :**
> **À l'Entrée :** La quantité reçue fait immédiatement AUGMENTER `TypeArticle.quantite_stock` et crée autant d'instances `Article` en base. **Au Bon de Sortie :** La validation réserve du stock mais NE DIMINUE PAS le stock physique.
> **À la Sortie Physique (CAME) :** C'est l'unique étape qui DIMINUE `TypeArticle.quantite_stock` et marque l'article comme sorti.

11. Banque de Questions de Soutenance — Niveaux 1 à 3

Q1 (Niveau 1 - Fondamental) : Pourquoi séparer TypeArticle et Article ?

Réponse : `TypeArticle` définit la catégorie générale (ex: "HP ProBook") et sa quantité globale en magasin, tandis qu'`Article` représente l'objet physique spécifique avec son propre numéro de série et son QR code. Cette distinction permet d'avoir à la fois une gestion globale pour les consommables et un suivi unitaire strict pour les biens de valeur.

Q2 (Niveau 1) : Qu'est-ce qu'une transaction atomique et pourquoi est-elle critique ici ?

Réponse : Une transaction atomique (`with transaction.atomic()`) garantit le principe du "Tout ou Rien". Lors d'une sortie de stock, le système doit créer une fiche `Sortie`, générer les `Mouvement` et réduire `quantite_stock`. Si le système plante au milieu, la transaction annule tout (Rollback) pour éviter une corruption de stock.

Q3 (Niveau 2 - Détaillé) : Pourquoi stock_reserve() est-elle calculée dynamiquement au lieu d'être un champ en base ?

Réponse : Pour garantir l'intégrité absolue des données. Un champ stocké risquerait d'être désynchronisé en cas d'erreur réseau, d'annulation manuelle ou de mise à jour concurrente. Le recalcul dynamique garantit que la réservation s'appuie toujours sur l'état exact et réel des `LigneBSortie` en cours.

Q4 (Niveau 3 - Analyse Technique) : Que se passerait-il si on retirait select_for_update() ?

Réponse : On s'exposerait à des "conditions de concurrence" (Race Conditions). Si deux magasiniers validaient simultanément une sortie pour 5 ordinateurs alors qu'il n'en reste que 6 en stock, les deux requêtes liraient un stock de 6 disponible et accepteraient la transaction, faisant chuter le stock physique à -4 (stock négatif invalide).

12. Banque de Questions de Soutenance — Niveaux 4 & 5

Q5 (Niveau 4 - Extension) : Comment ajouter une notification par Email à la validation d'un bon ?

Réponse : Il faut intervenir dans `valider_ligne()` dans `services.py`. Une fois la ligne validée et le statut du bon recalculé à `valide`, on utilise le module `django.core.mail.send_mail()` à l'intérieur de la transaction pour notifier le demandeur et l'agent CAME.

Q6 (Niveau 5 - Cas Limite) : Comment le système réagit-il si la quantité demandée est supérieure au stock disponible ?

Réponse : Au niveau backend, `autoriser_ligne()` lève une exception `ValidationError` si `quantite > disponible`. Au niveau frontend, le formulaire DSID intercepte le stock en amont, affiche une alerte et force le DSID à réajuster la quantité autorisée (ou à mettre 0 pour refuser la ligne avec motif obligatoire).

13. Guide de Débogage & Diagnostics Fréquents

Erreur / Message	Cause Origine	Solution Technique
`Cette ligne a déjà été traitée par le DSID`	Tentative de ré-autoriser une ligne dont l'état n'est plus `demandee`.	Filtrer les lignes dans React sur `l.etat_ligne === 'demandee'` avant de boucler sur l'API.
`403 Permission Denied`	L'utilisateur connecté n'a pas le rôle requis (ex: demandeur exécutant une sortie).	Vérifier le rôle de l'utilisateur dans `ROLES_AUTORISATION` / `ROLES_SORTIE` dans `services.py`.
`Stock insuffisant lors de la sortie`	Une sortie a décrémenté le stock physique entre l'autorisation et la livraison.	Vérifier que le stock réservé prend bien en compte toutes les lignes `autorisee` et `validee`.

14. Glossaire des Termes Techniques du Projet

Terme	Définition & Application dans le Projet
DRF (Django REST Framework)	Surcouche à Django facilitant la création d'APIs Web RESTful sérialisant les modèles en JSON.
ORM (Object-Relational Mapping)	Technique qui mappe les tables de la base de données vers des objets Python (`models.Model`).
Payload JSON	Structure des données au format texte transitant dans le corps des requêtes HTTP REST.
Hooks React (`useState`, `useMemo`)	Fonctions permettant d'attacher un état local et des valeurs calculées aux composants React.
CAME / DSID / DCA	Rôles d'utilisateurs métiers : DSID (Autorisation), DCA (Validation), CAME (Sortie Magasin).

15. Synthèse pour le Jour de la Soutenance

Fiche Mémo de Révision Rapide :

1. **Intégrité Métier :** Aucune mise à jour directe en base depuis le front ; tout transite par les fonctions encapsulées de `services.py`.
2. **Gestion de Concurrence :** Utilisation systématique de transactions atomiques (`transaction.atomic`) et de verrous SQL (`select_for_update`).
3. **Double Gestion d'Articles :** Gestion par lot/quantité globale pour les consommables vs gestion unitaire individualisée pour le matériel lourd.
4. **Workflow d'Approbation :** Séparation stricte des responsabilités (Demande -> Autorisation DSID -> Validation DCA -> Sortie CAME).